

Day 10 - Typography, Color, and Accessibility

Day 10 — Typography, Color, and Accessibility

Objectives

- Choose and apply fonts properly, including a sensible fallback strategy
- Control readability with `line-height` and `letter-spacing`
- Understand the three common color formats and when each is useful
- Understand color contrast and why it's a real, measurable accessibility requirement, not a vague suggestion
- Tie together everything you've learned about `alt` text, labels, and focus styles into a coherent picture of what "accessible" actually means

Fonts and fallback stacks

```
body {  
  font-family: "Helvetica Neue", Arial, sans-serif;  
}
```

`font-family` accepts a **comma-separated list**, tried in order: the browser attempts to use the first font; if that specific font isn't installed on the visitor's device, it falls back to the next one in the list; and so on. The very last entry should always be a **generic family name** (`sans-serif`, `serif`, or `monospace`) — this guarantees that even if none of your specifically-named fonts are available, the browser still picks *some* reasonable font of the right general category, rather than falling back to something wildly inappropriate.

- `sans-serif` — fonts without small decorative strokes ("serifs") at the ends of letters (Arial, Helvetica) — generally considered cleaner and more common for screens and UI text.
- `serif` — fonts with those small decorative strokes (Times New Roman, Georgia) — often used for long-form reading text, evoking a more traditional, print-like feel.
- `monospace` — every character takes up exactly the same width (Courier, Consolas) — used for displaying code, exactly like the code blocks throughout this very course.

Web fonts — using a font that isn't already installed on the visitor's device

The fonts above are all common ones already installed on most computers. To use something more distinctive, you load a **web font** — a font file the browser downloads as part of loading your page. The most common, beginner-friendly way to do this is via **Google Fonts** (a free, hosted library):

```
<head>  
  <link rel="preconnect" href="https://fonts.googleapis.com">  
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">  
</head>  
  
body {  
  font-family: "Roboto", Arial, sans-serif;
```

```
}
```

Notice you still include a fallback stack after your web font — if the web font fails to load for any reason (a slow connection, the service being briefly unreachable), the page still displays reasonably rather than showing completely unstyled system fonts unexpectedly.

`line-height` and `letter-spacing` — controlling readability

```
p {  
  line-height: 1.6;      /* the vertical space between lines of text, as a MULTIPLE of the font size */  
  letter-spacing: 0.02em; /* extra horizontal space between individual characters */  
}
```

line-height: 1.6 means each line takes up 1.6 times the font size in vertical space — a genuinely important, easy-to-overlook readability factor: text with lines packed too tightly together (**line-height: 1** or lower) is measurably harder to read, especially for longer paragraphs, because your eye has a harder time tracking from the end of one line to the start of the next. A value between about 1.4 and 1.6 is a common, comfortable default for body text; headings often use a tighter value since they're usually short and don't have the same "tracking between lines" problem. **letter-spacing** is used far more sparingly — usually for small, all-caps text (like a label or button), where slightly increased spacing can improve legibility.

Color formats — hex, rgb, and hsl

```
.box {  
  color: #3366cc;          /* hex: 6 hexadecimal digits -- 2 each for red, green, blue */  
  color: rgb(51, 102, 204); /* rgb: the same color, as literal 0-255 red/green/blue values */  
  color: hsl(220, 60%, 50%); /* hsl: hue (0-360 degrees), saturation %, lightness % */  
}
```

All three describe the exact same underlying color — they're just different ways of writing it down. **Hex** is the most compact and most common in real-world CSS, but it's genuinely hard for a human to reason about what color a random hex code actually represents just by reading the digits. **hsl** (hue, saturation, lightness) is often far more intuitive to work with directly, especially when you want a *family* of related colors: keep the hue the same and just change the lightness percentage to get lighter/darker versions of the exact same color, or change the hue alone (0=red, 120=green, 240=blue, moving around a color wheel) to shift to a completely different color while keeping the same saturation/lightness "feel." Many developers reach for **hsl** specifically when they need to programmatically generate a set of related shades or when tweaking a color live in DevTools, since sliding one clearly-named number (hue, saturation, or lightness) is more predictable than guessing at hex digits.

Add transparency to any of these with an added alpha value: **rgba(51, 102, 204, 0.5)** or **hsla(220, 60%, 50%, 0.5)** (the **a** stands for "alpha," where **1** is fully opaque and **0** is fully transparent) — or, in modern CSS, you can add a fourth value directly to plain **rgb()**/**hsl()** without needing the separate **a** variants at all.

Color contrast — a real, measurable accessibility requirement

Text needs sufficient contrast against its background to be readable — not just for visually impaired visitors, but for anyone using their phone outdoors in bright sunlight, or anyone with even mild, common color vision

differences. This isn't a vague design preference; it's governed by an actual published standard, **WCAG** (Web Content Accessibility Guidelines), which specifies minimum **contrast ratios** — a mathematically-calculated measure of how different two colors are in perceived brightness. The commonly-cited baseline (WCAG AA) requires a contrast ratio of at least 4.5:1 for normal body text, and at least 3:1 for large text (headings).

You do not need to calculate this ratio by hand — browser DevTools will do it for you: in the Elements panel, click on any text element, find its `color` value in the Styles panel, and clicking the small color swatch next to it typically shows the calculated contrast ratio directly, along with whether it passes the AA (and stricter AAA) thresholds. Get in the habit of checking this for your text/background color combinations, especially anything with a light gray text on a white background — a genuinely common, easy mistake that looks fine on your own well-lit monitor but fails contrast requirements and is hard for many people to read.

Pulling it all together: what "accessible" actually means, in concrete terms

You've encountered several individual accessibility-related practices scattered across the last several days — here they are, gathered into one coherent picture, since together they form the actual, practical definition of an accessible page:

- **Meaningful `alt` text** on every informative image (Day 1) — so a screen reader user knows what an image shows.
- **Properly connected `<label for>/id`** on every form input (Day 2) — so both screen readers and simple click targeting work correctly.
- **Semantic HTML elements** (`<nav>`, `<main>`, `<button>` instead of a styled `<div>`, and correct heading levels) (Days 1-2) — so a screen reader user can navigate your page's structure directly, and so keyboard users get correct default behavior (a real `<button>` is focusable and triggerable with Enter/Space automatically; a `<div>` styled to look like a button is not, unless you do significant extra work to make it so).
- **Visible `:focus` styles**, never removed without an equally visible replacement (Day 9) — so a keyboard-only user can always see exactly where they currently are on the page.
- **Sufficient color contrast** (today) — so text is genuinely readable for the widest possible range of visitors and viewing conditions.

None of these are exotic, specialized techniques reserved for some separate "accessibility mode" of a website — they're just the ordinary, correct way of using HTML and CSS, and building this awareness in **now**, in your first two weeks, means you'll simply build things correctly from the start, rather than needing to retrofit accessibility onto a project later (which is dramatically more work than doing it correctly the first time).

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO --> /* TODO */` markers, and check your work against `CHECKLIST.md`.